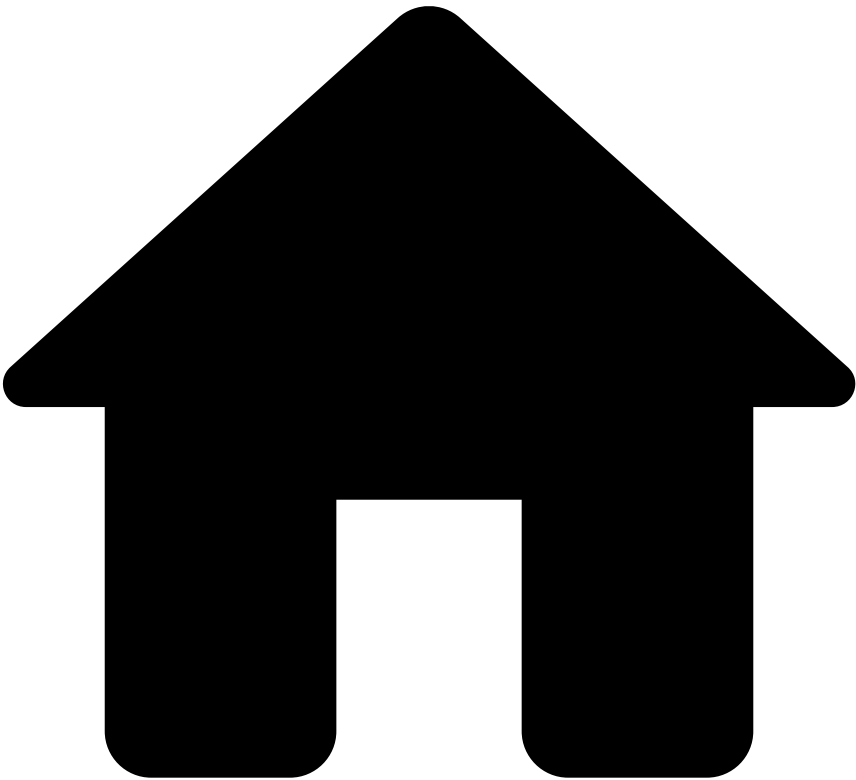


•



- [Core Concepts](#)
- [Workflows](#)
- [Railgun Engine](#)
- Metrics in Railgun

On this page

Metrics in Railgun

Was this page helpful? 

A [metric](#) is the smallest unit of work in Railgun. It is defined by a time window, a set of `group by` fields (optionally coupled with a matching set of `lookup by` fields), an aggregation function, and an optional filter.

As an example, the metric *"total amount spent by each card over the last month"* defines a window (*last month*), a `group by` field (*card*) and an aggregation (*total amount*).

Metric components

Windows

The time window that is defined for a metric determines the set of events that are used to compute the metric value. The events that are within the window can change over time as new events arrive to Railgun, some may be added to the window, while others, that were already in the window, may leave/expire from the window. Railgun ensures that the metric values are kept up to date as events enter and leave the window.

In stream processing, there are many types of windows that can be defined. Below, we list the window types that are supported by Railgun.

Sliding Windows

To define a sliding window we just need to specify its size (as a time interval). The window will hold all the events received within this time interval. For example, a sliding window with a 10 minutes size will contain only the events received during the past 10 minutes.

sliding window

The metric value is updated as soon as new events enter the time interval defined by the window, and/or old events are no longer within this time interval i.e. as the logical time in Railgun advances. Do note that, as Railgun processes events and the logical time in Railgun advances, it could happen that the window becomes empty (without events) if indeed there are no events for the time frame that is defined by the window.

In Railgun, the lower bound of the window (older events) is exclusive, while the upper bound (more recent events) is inclusive.

Landmark Windows

Unlike sliding windows, landmark windows expire all events in the window at once at well-defined moments in time. These instants are defined by a reset expression using the `reset(unit:_)` syntax.

New events are added to the window until the time reaches the trigger instant defined by the reset expression. At that moment, the window resets and starts to refill as new events arrive.

For instance, using `reset(day:_)` means the window resets every day at 00:00.

landmark window

Landmark windows have some limitations in Railgun that are important to point out:

1. The reset instant depends on both the expression and the system time zone. Railgun can be configured with a specific time zone for all windows.
2. The expression must define a **fixed interval** at which the window resets. Non-constant frequencies (such as specific weekdays or day-of-month ranges) are not supported, since they are also used by Railgun to configure time intervals for backfills and data retention, which require a predictable, constant repeating frequency.

The following table lists the supported expressions for landmark resets:

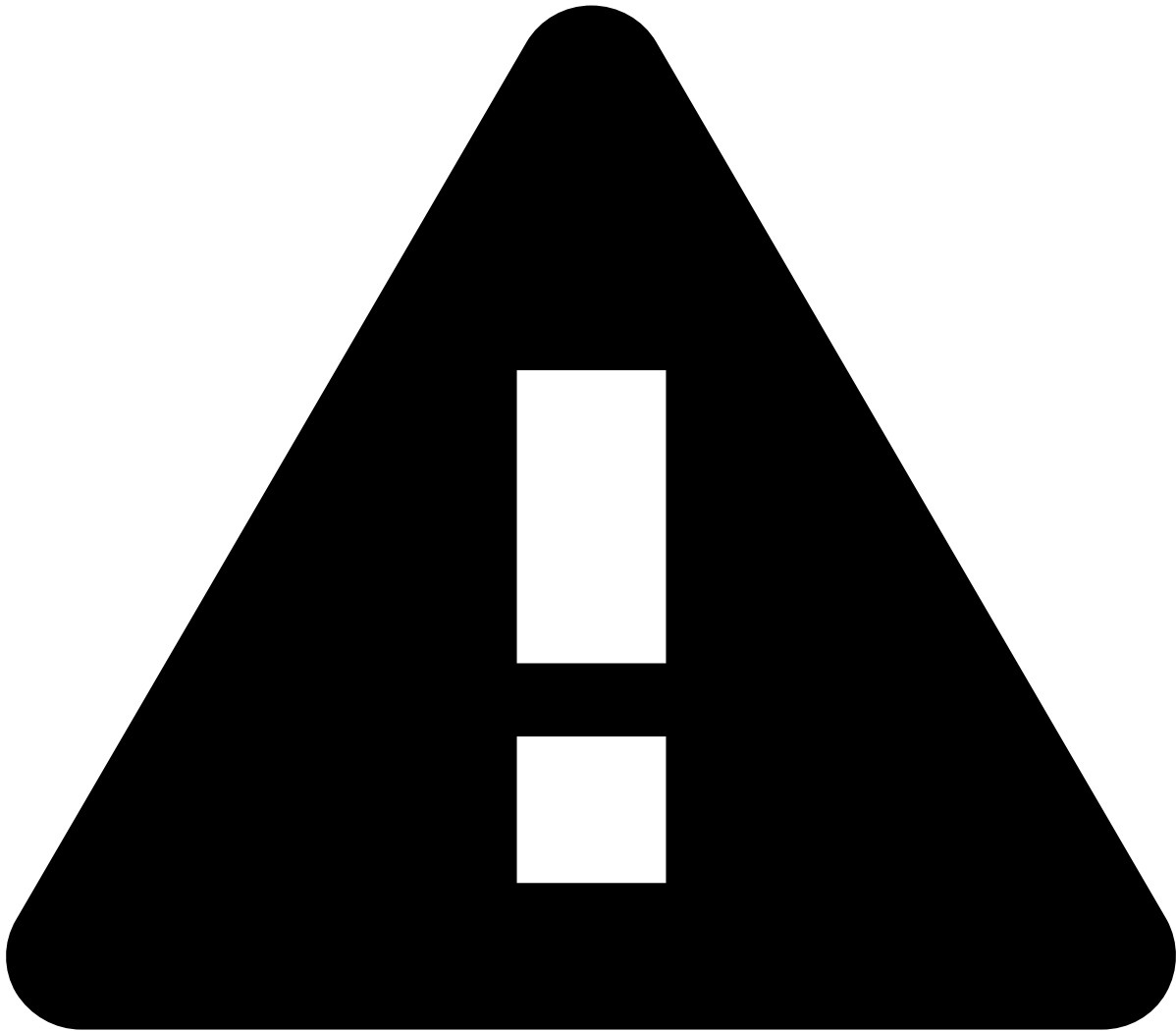
Expression	Reset frequency
<code>reset(day:_)</code>	Daily at 00:00
<code>reset(hour:_)</code>	Every hour at the start of the hour
<code>reset(min:_)</code>	Every minute at the <code>:00</code> second mark



Default values

When a unit is omitted from the expression, Railgun assumes the value is `0` for smaller units and `_` (every) for larger units. For example, `reset(day: _)` automatically assumes `hour: 0`, `min: 0`, and `sec: 0`.

3. The lower bound of the window (older events) is inclusive, while the upper bound (more recent events) is exclusive.
4. Currently, Railgun does not support using landmark windows together with:
 - [Count Distinct aggregations](#) being used in a metric inside those windows;
 - [Output Every](#);
 - [Snapshot Metrics](#);
 - [Delays](#) specified for those windows.



No support for tumbling windows

[Tumbling windows](#) are similar to [landmark windows](#). The difference is that tumbling windows are only defined by their size, and they tumble/reset every time they reach that size. Railgun only supports landmark windows because tumbling windows don't provide a practical advantage over them.

Additionally, tumbling windows may not yield the result that users are expecting. For instance, the tumbling of the windows starts counting at the origin of the Unix time. It is a common misunderstanding that a tumbling windows of 1 day would reset every day at 00:00, but this does not happen due to adjustments such as leap seconds.

Adding Delays to Windows[↗](#)[📄](#)

In addition to the window types that are listed above, it is possible to add a delay to a window. For instance, Railgun allows to define a sliding window of one day with a delay of one day, as a way to compute the metric based on the events only from the previous day.

delayed windows

Non-windowed Metrics[↗](#)[📄](#)

Railgun makes it possible to define metrics that are not bound to a specific time window. These metrics include all events available in the system, and are updated as new events arrive. Such windowless metrics are useful for computing aggregations over the lifetime of the associated entities; for example, the total amount spent by a customer since the beginning of recorded data, or the first time a customer executed a certain type of transaction.

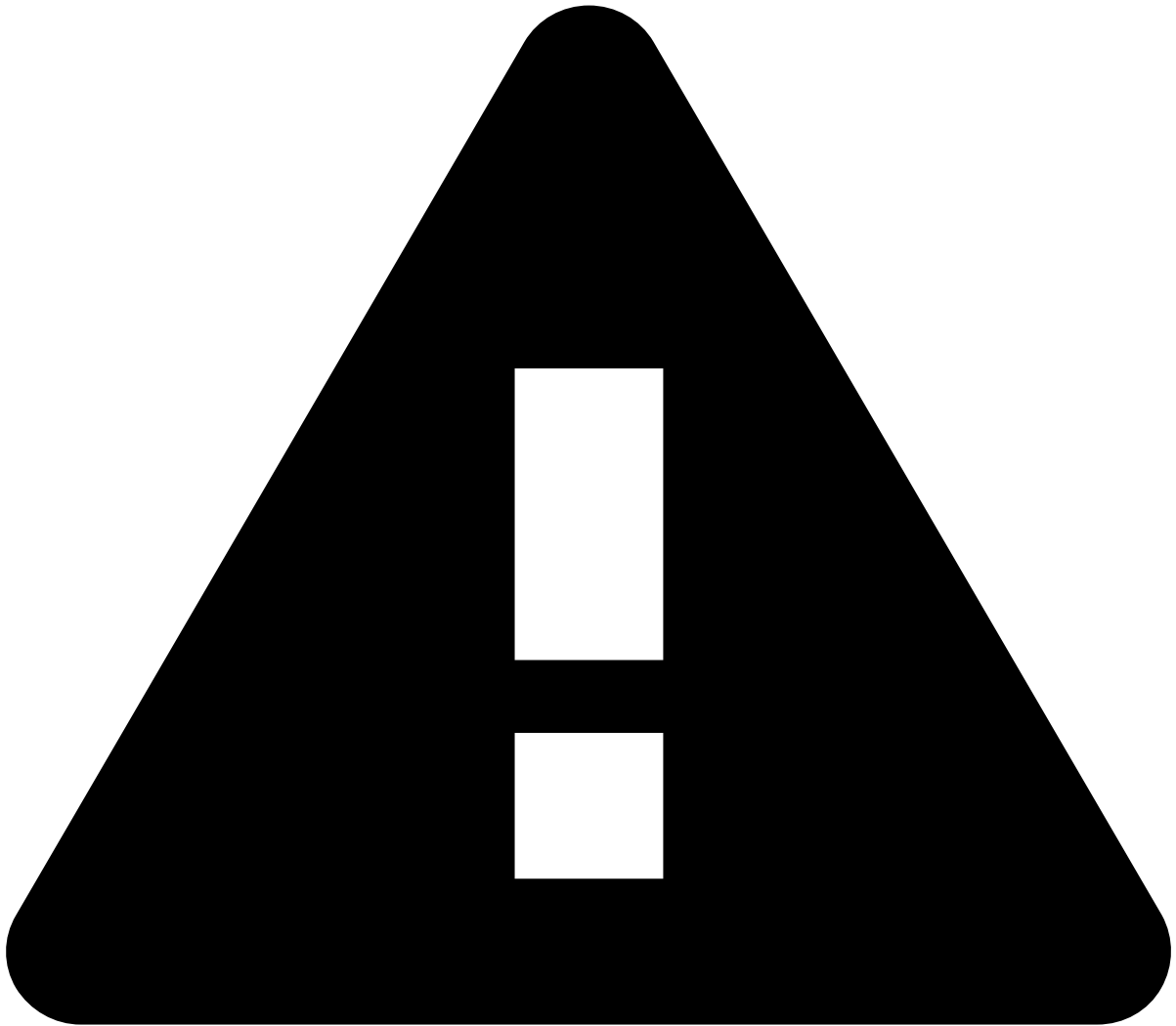


info

Non-windowed metrics can only use a limited set of aggregations. Whereas metrics with sliding time windows can use all aggregations, non-windowed metrics are limited to the `Max` and `Min` aggregations.

When first creating a non-windowed metric, the resulting [backfill](#) will process all events from the earliest available records until the current time. The value of the metric will then be continually updated as new events arrive, while keeping into account the values previously calculated, even when these values result from events that later expire from the maximum data lookup period due to the application of our data retention policies.

For example, a metric that computes the total amount transacted by a customer towards a specific beneficiary will seamlessly accumulate the value as new transactions are processed; older transactions that are no longer available in the system will not decrease the value of the metric, since Railgun preserves an internal state for metric values as part of its operation.



warning

If the definition of a non-windowed metric is updated, the metric will be backfilled again, but the new value will not take into account events that are no longer available due to the data retention policy.

Group By

Similar to SQL, we can define a (non-empty) set of fields in the events to partition windows into groups and compute aggregations individually over these groups. As an example, we can group by the country field to do a count of transactions per country.

group by

The metric then returns the aggregated value (count of transactions) of the group whose key matches the grouping field (country) of the current event.

Lookup By

Optionally, it is possible to specify a set of alternative lookup fields, compatible with the grouping fields, so that the metric will return the aggregation for the group matching the `Lookup By` fields of each event, instead of the group matching the `Group By` fields.

This overrides the default behavior of the `Group By` operator, and allows the retrieval of the aggregation for any of the partitions, as long as the schema of the data stream contains additional fields that can hold the same values as those of

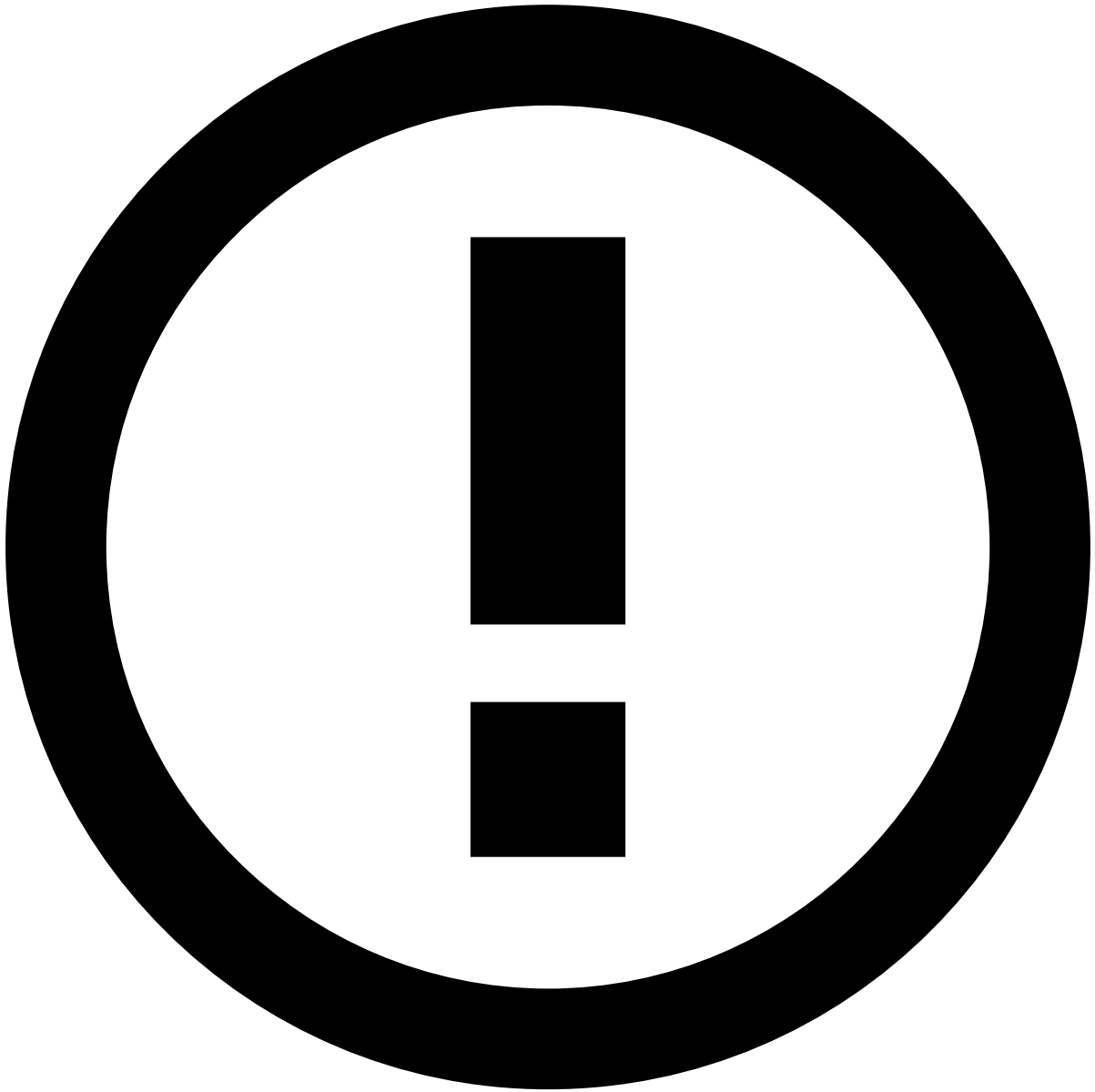
the grouping fields (e.g. a *sender_id* and a *recipient_id*, or a *origin_country* and a *destination_country*, etc.) and can therefore be specified in group-by/lookup-by pairs.

This expands the range of insights the metric can provide, such as determining the total amount received by the account that is sending money in a transaction.

A specific example where such lookup metrics might be useful is to calculate an account's recent running balance (net flow of money) within a rolling period of time, such as the past week prior to each transaction. To do so, one might set up the following structure:

- A regular group-by metric to get the total sent by the transaction's sender:
`{ groupBy: [$sender], aggregation: sum($amount), window: 7 days }`
- A lookup metric to get the total *received* by the transaction's sender:
`{ groupBy: [$receiver], lookupBy: [$sender], aggregation: sum($amount), window: 7 days }`
- A [map operator](#) that subtracts the former from the latter (`total_received_7d - total_sent_7d`), thus combining the two metrics into a third [derived field](#) providing the sender's current balance.

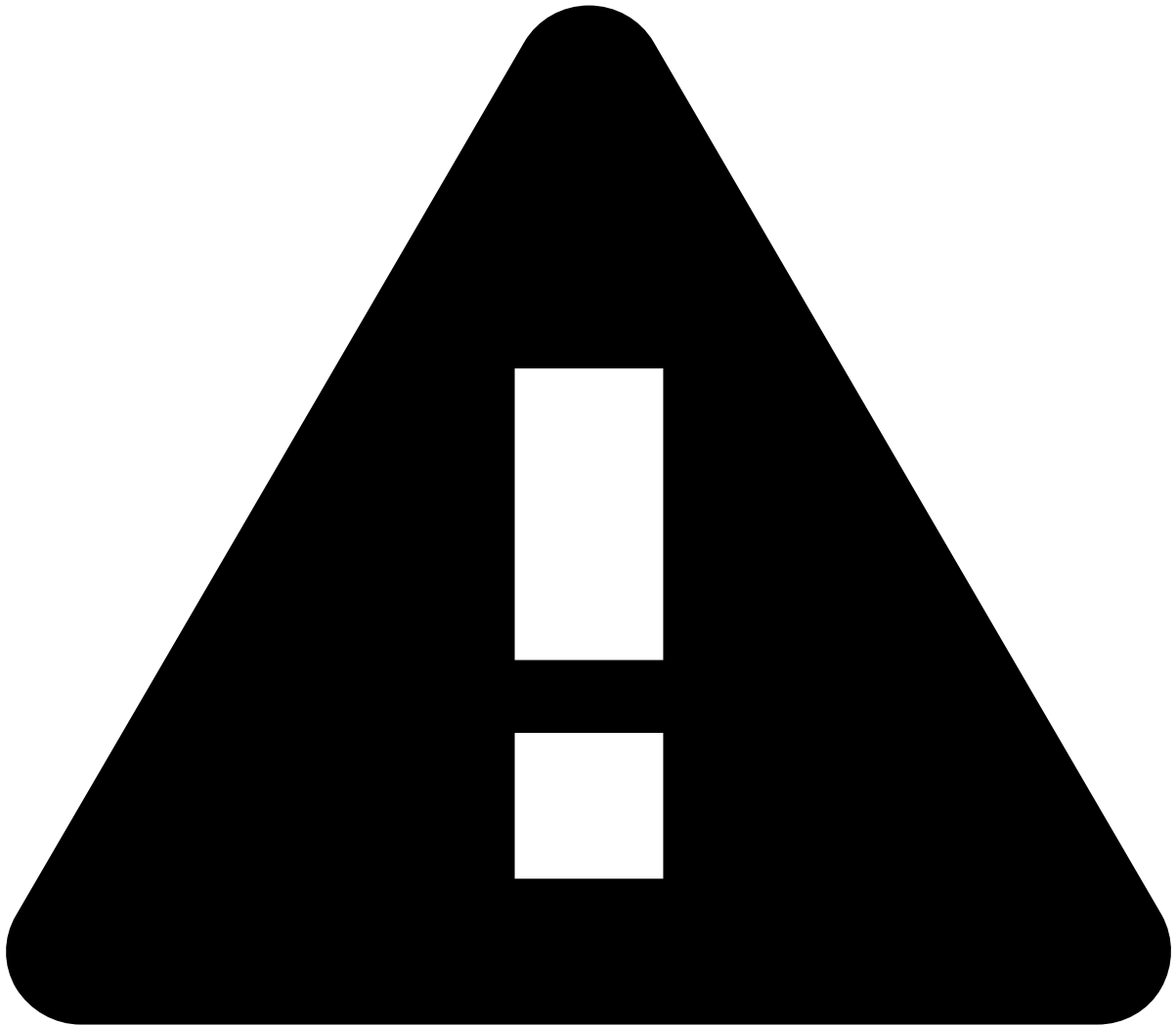
The [Using Lookup Metrics](#) page describes how to implement this example in detail.



Compatibility of the lookup and grouping fields

For the lookup to work, the lookup fields must be compatible both in number and [data type](#) with the grouping fields.

Refer to the [Using Lookup Metrics](#) page to learn how to create lookup metrics in Pulse.



warning

Lookup By fields are not supported in [cross-channel metrics](#).

Aggregations

Aggregations allow performing computations over the multiple events that are in the metric window at a given point in time.

Count

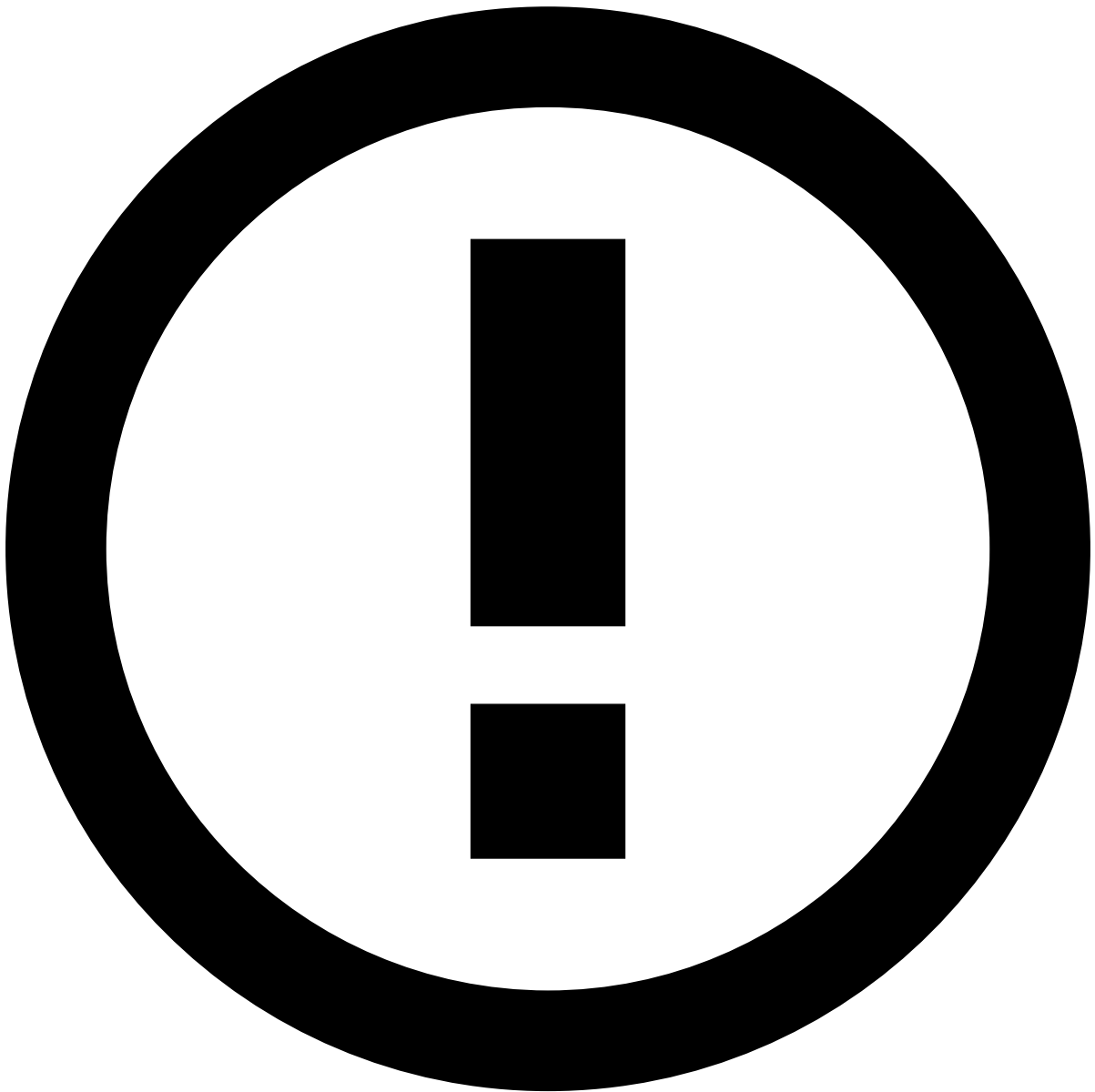
Returns the count of events.

- **Initial value:** 0
- **Return type:** long

Sum

Returns the sum of values of a given field in the events.

- **Supported operand types:** integer, long, float and double
- **Initial value:** null
- **Return type:** long (if operand is an integer or long) or double (if operand is a float or double)



info

The sum is computed as a plain Java sum. Railgun does not use algorithms such as [Kahan's summation](#) to reduce numerical errors when summing float or double values.

Average ⓘ ⓘ

Returns the average value of a given field in the events.

- **Supported operand types:** integer, long, float and double
- **Initial value:** 0
- **Return type:** double

Standard Deviation ⓘ ⓘ

Returns the standard deviation of the values of a given field in the events. The standard deviation is computed in an incremental way by using the [Knuth-Welford's online algorithm](#).

- **Supported operand types:** integer, long, float and double

- **Initial value:** 0
- **Return type:** double

Max / Min

Returns the maximum or minimum value that was observed for a given field in the events. For sliding windows, Railgun computes this aggregation by using the [sliding window minimum/maximum algorithm](#).

- **Supported operand types:** integer, long, float and double
- **Initial value:** null
- **Return type:** same type as the operand



Performance guideline(s)

The algorithm that is used is designed to be as computationally efficient as possible, taking into account that the state is persisted on disk. The algorithm keeps a sorted deque of values that will eventually become the current maximum/minimum values as events expire from the window.

However, if the values of the operand that is used in the aggregation are monotonically increasing (for `min`), such as a timestamp, or monotonically decreasing (for `max`), the underlying deque that is used will keep increasing in size because all values will eventually become the current maximum/minimum. As a consequence, latencies will increase as this deque grows (due to the cost of serializing/deserializing the deque) and may eventually hit a system limitation.

For recommendations on how to handle these kinds of metrics, refer to the [Usage of min\(\) and max\(\) aggregations](#) section in the "Best Practices for Metrics in Railgun" page.

Prevâ

Returns the previous value that was observed for a given field.

- **Supported operand types:** boolean, string, integer, long, float and double
- **Initial value:** null
- **Return type:** same type as the operand

Lastâ

Returns the last value that was observed for a given field. The use of this aggregation is **only useful when the metric uses a filter** where only a subset of events can update the aggregation (in this case it would return the value of the field for the last event that matched the filter). Otherwise, the value returned by this aggregation is the value of the field in the event that is currently being processed, which makes the metric useless.

- **Supported operand types:** boolean, string, integer, long, float and double
- **Initial value:** null
- **Return type:** same type as the operand

Count Distinctâ

Returns the **exact** count of distinct values of a set of fields (one or more fields) in the events.

As an example, if we observe the following values for the fields (country,amount) :

Event	Country	Amount
1	Portugal	100
2	Spain	200
3	Spain	300
4	Portugal	100

...then, a distinct count with the fields (country,amount) would return 3; a distinct count with field (country) would return 2; and a distinct count with field (amount) would return 3.

- **Supported operand types:** boolean, string, integer, long, float and double
- **Initial value:** 0
- **Return type:** long



Performance Guideline(s)

Even though there isn't a strict limit to the amount of operands that can be used in the aggregation, a maximum of 2 is recommended. The size/space of the values of the operand fields may also have an impact on the overall performance. Larger values imply larger keys in RocksDB, which reduces the amount of data that the system can cache in memory.

The cardinality of distinct values may also have a performance impact. More distinct values increases the likelihood of having to read values from disk (instead of reading them from memory), which has a performance impact.

Filters

A filter is a boolean expression against which events are evaluated to decide whether a given event should (if evaluates to true) or not contribute to the metric value. The filter expression **is optional** and is defined as a [PQL expression](#).

Functions available in filters

In the PQL expression that defines the filter, there are some utility functions that are available and can be used. As an example, the `StringUtils.toUpperCase` function can be used in a filter: `StringUtils.toUpperCase(country) == "PORTUGAL"` .

Railgun doesn't allow for clients to plug in their own functions. Instead, it supports the following commonly used functions:

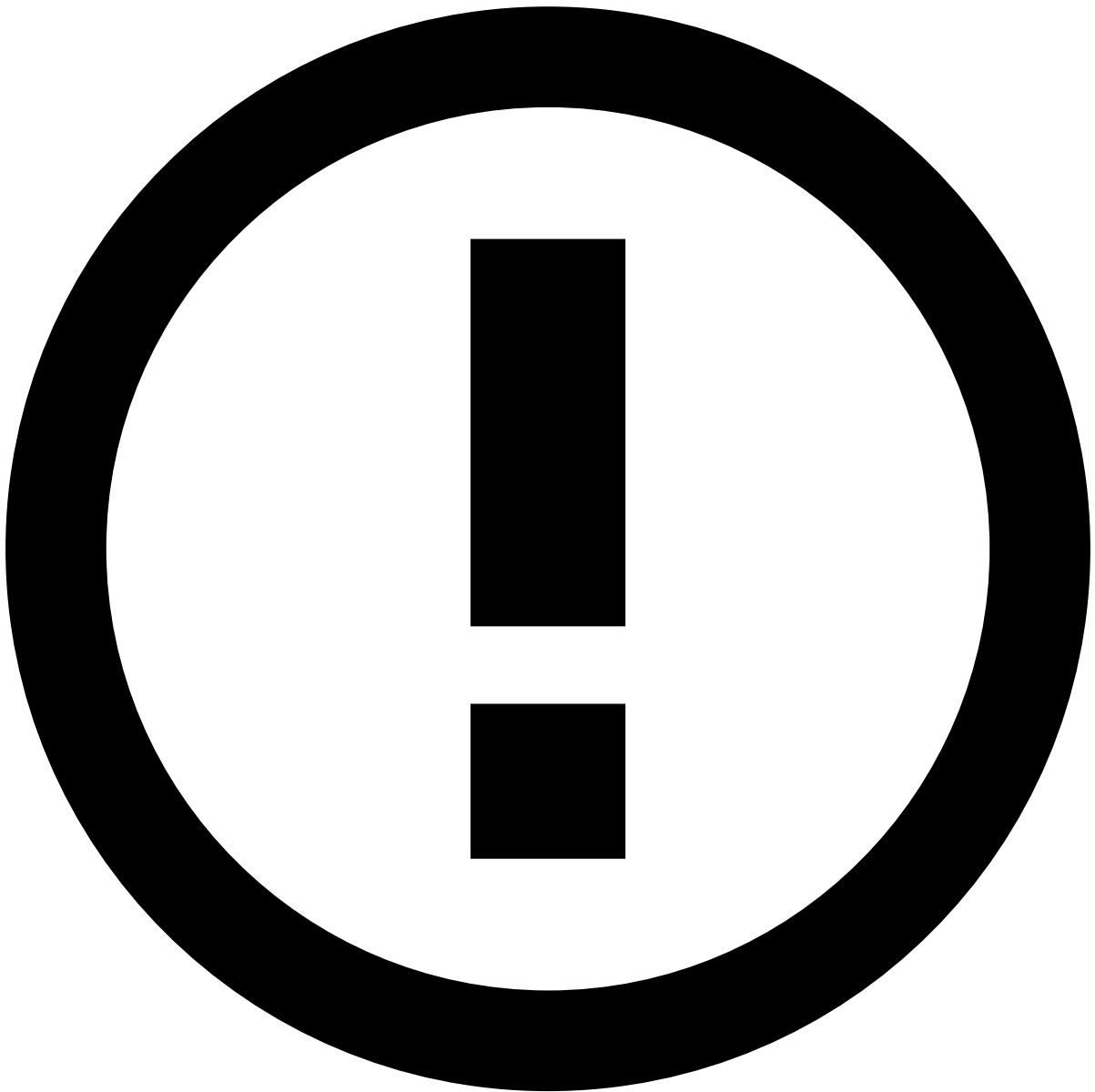
Name	Fully Qualified Name
StringTools	com.feedzai.commons.udfs.StringTools
StringUtils	org.apache.commons.lang3.StringUtils
Collections	com.feedzai.pulse.fraud.udf.Collections
Country	com.feedzai.pulse.fraud.udf.Country
Crypto	com.feedzai.pulse.fraud.udf.Crypto
Location	com.feedzai.pulse.fraud.udf.Location
Maths	com.feedzai.pulse.fraud.udf.Maths
Strings	com.feedzai.pulse.fraud.udf.Strings
Time	com.feedzai.pulse.fraud.udf.Time
Zip	com.feedzai.pulse.fraud.udf.Zip
CollectionUDF	com.feedzai.pulse.pkernel.runtime.CollectionUDFs

More functions can be added to this list but must be first evaluated by the Railgun maintainers.



danger

This feature is maintained to facilitate Railgun adoption. It will eventually be deprecated, and as such it has limited support.



info

This limitation only applies to the usage of functions in metrics' filters. **It does not apply** to the usage of functions (UDFs) in Pulse map operators.

Fallback Value📄

When defining a metric in Pulse, users may choose to define a **fallback value** for the metric. When this fallback value is defined, Pulse will automatically coalesce `null` metric values to the value that the user defines.

fallback-value.png

There are two occasions where metrics calculated by Railgun can return a `null` value:

1. **When the metrics' aggregation can output a null value.** As an example, a "Last" aggregation can return `null` if there is no last value (i.e. no event is contributing to the metric), or if the last value exists but is null. But a "Count" will never output a `null` as it will return 0 if no event is within the metric window. See the [default values for each specific aggregation](#) that Railgun supports.

2. **While the metrics is being backfilled.** For example, when a metric was just created, or its definition was updated, it will output a `null` value until its backfill successfully completes.

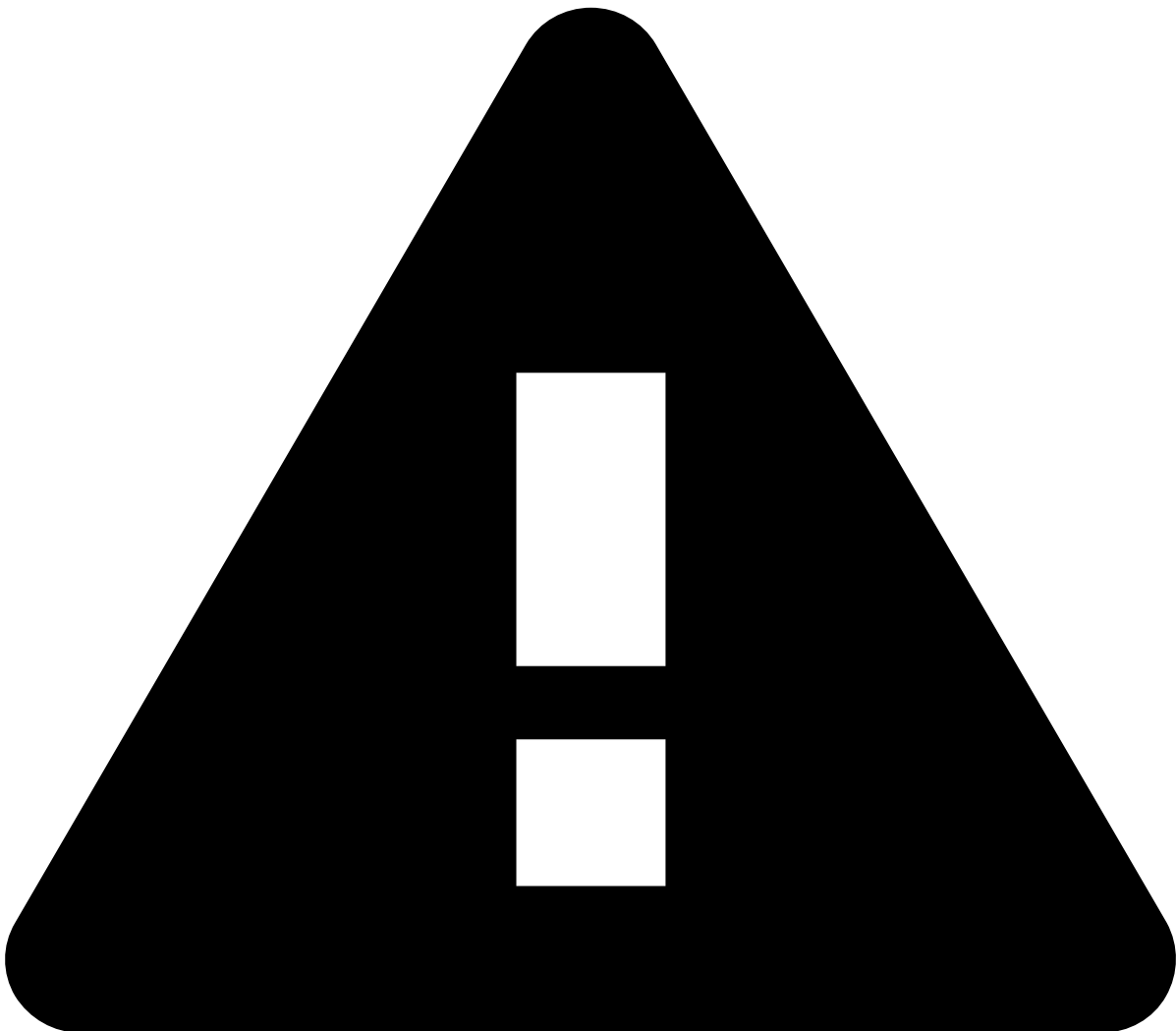
This means that any downstream logic in Pulse workflows must take into account that the metrics' values can be `null`. The possibility to define a fallback value for these cases makes the workflow's behavior more user-friendly and less error-prone.

By default, the fallback value is enabled with a predetermined value that is automatically derived from the metric output type:

- If the output type is a **String**, null values will be coalesced to an **empty string** (`""`).
- If the output type is an **Integer or floating point**, null values will be coalesced to **0**.
- If the output type is a **Boolean**, null values will not be coalesced, remaining as **null**.

The fallback value can also be **set to a custom value**, by entering the desired value in the input box next to the "Enable" checkbox:

custom-fallback-value.png



Custom fallback values

Custom fallback values must be appropriate for the metrics' data type; otherwise the validations will fail. If the input box is left empty, then a null or empty string (for String data types) will be used for the coalesced value.

It is also possible to **disable the fallback value** (by unmarking the "Enabled" checkbox). In this case, no coalescing will be applied. If the metric value is null, then it will be forwarded as null to the remainder of the workflow.

disabled-fallback-value.png

Output Every (Scheduled Metrics equivalent)

This is a feature that can be optionally enabled and that causes the metric value to only be updated/refreshed at a given frequency, defined by a reset expression. It is as if Railgun periodically takes a snapshot of the metric value at a given moment in time, and outputs that value until the next snapshot is taken.

The update frequency is defined by a reset expression (see [Landmark Windows](#) for supported units).

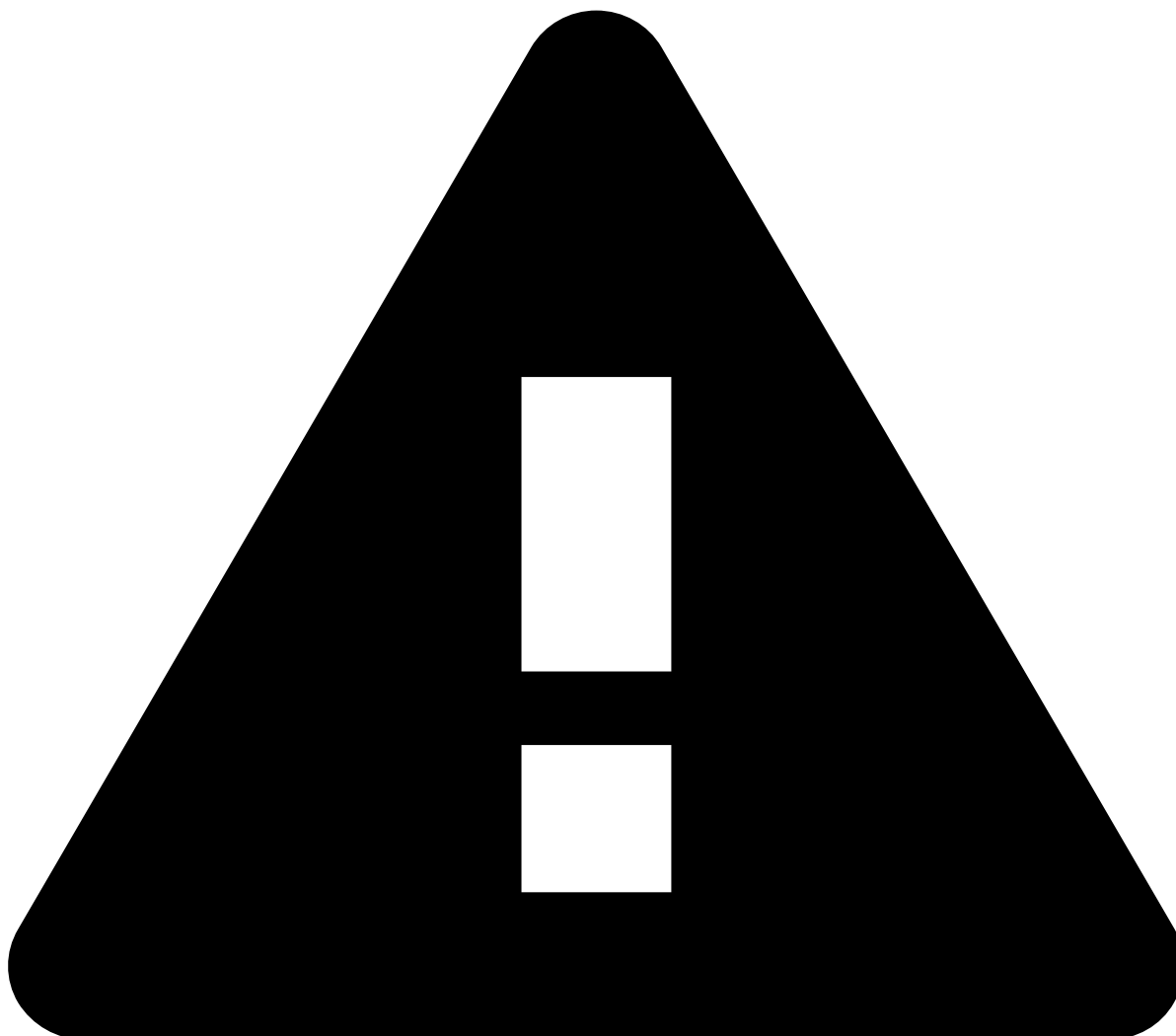
output every



Rationale for `output every`

This feature was added so that users can have metrics that semantically are equivalent to [scheduled metrics](#), i.e. metrics that only refresh/update with a given frequency, and that are computed by running Spark jobs against the Event Storage.

Performance-wise, the metric continues to be updated in real time â its internal state continues to be updated one event at a time.



No support for the `first` aggregation

The `first` aggregation is not supported due to performance reasons. To compute the first aggregation Railgun would need to keep a queue of all event field values that can potentially become the output of the aggregation. This means that the queue would grow linearly with the number of events in the window.

For the same reason, it is not a good idea to use the minimum aggregation over a timestamp.

Snapshot Metrics

This feature is similar to [Output Every](#) metrics, with the caveat that instead of storing a single snapshot that is then replaced by a more recent one, Railgun allows a pre-defined number of snapshots to be stored and read.

As with Output Every, the frequency of the snapshots is defined by a reset expression; the difference is that Snapshot metrics also require the user to define a snapshot retention. For example, using `reset (day:_)` with a retention of 4 snapshots allows the user to read data from the past 4 days.



info

Snapshot metrics can be used as a replacement for the Output Every metrics by configuring a Snapshot metric with a retention value of 1.

However, keep in mind that Output Every metrics are more space conservative and will have better performance.

Snapshot metrics

Since we store multiple snapshots we must be able to query these snapshots. To achieve this, users can inject an event with a timestamp in the past and Railgun will look up the closest previous snapshot available.

The example below shows how injecting events in a certain day will return the snapshot stored in the previous day. If a snapshot is no longer available due to being expired, Railgun will return a 'null' value in response.

Snapshot metrics query

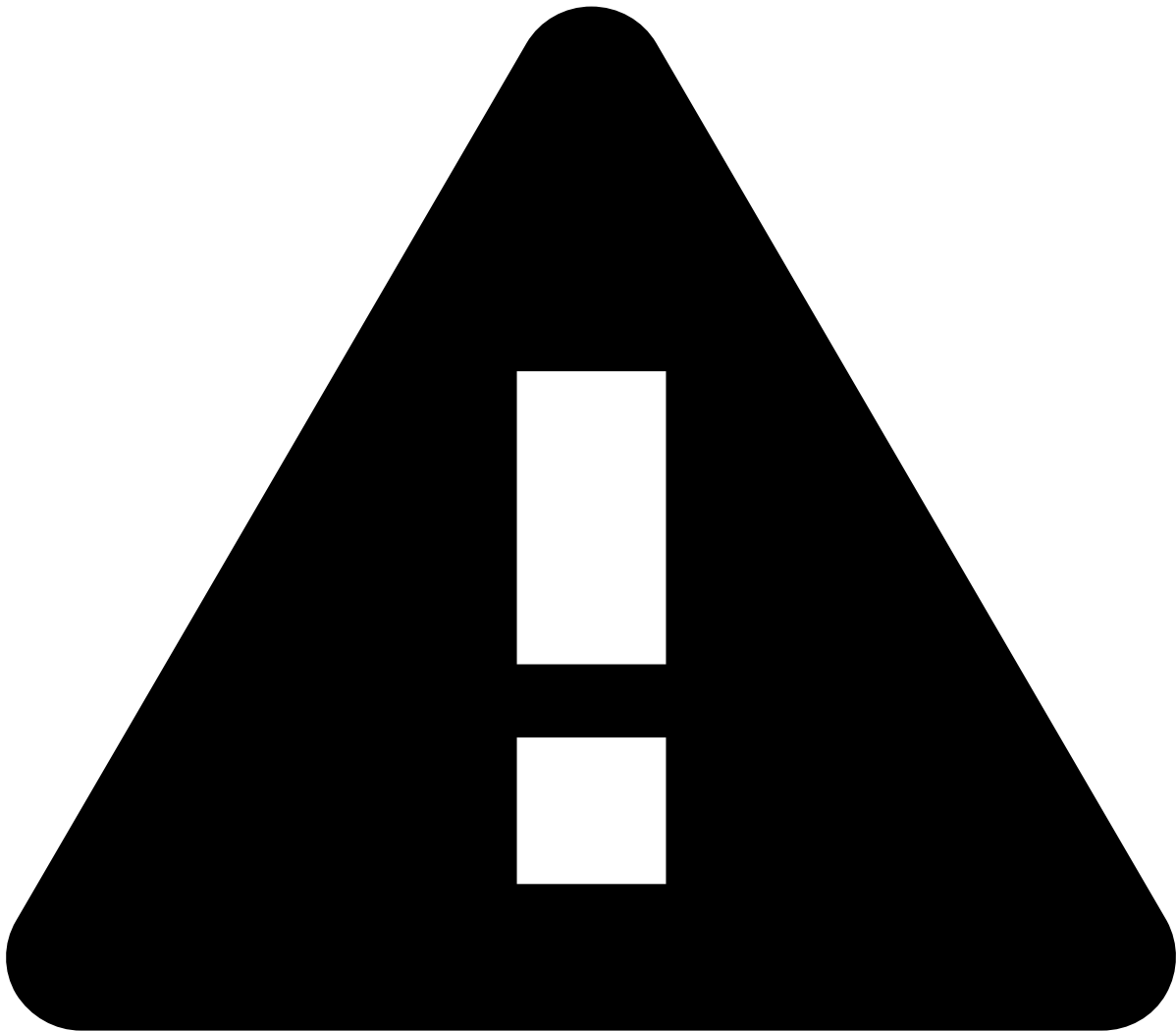
Cross-channel metrics⚠️

A **cross-channel** metric is a specific case of an [omnichannel metric](#) that only has **2 channels**, in which:

- Only one **Secondary** channel can contribute to the cross-channel metric.
- Only one **Main** channel will fetch the metric value, created and updated on the **Secondary** channel.
 - The **Main** channel **cannot contribute** to the cross-channel metric.

Note that each channel has its own detection strategy, and it's implemented in a Pulse application [workflow](#).

See the [Using cross-channel metrics](#) page to know how to create Cross-channel metrics in Pulse.



caution

Do not confuse these metrics with [omnichannel metrics](#), which are a superset of metrics that can be used in multiple channels, and can have **more than one** channel contribute to them. Also note that Railgun doesn't support omnichannel at the moment.